

Moving from Conda to uv

A plain-language guide to uv for people who have only used Conda (or nothing at all): what the files mean, the commands you need, how to update things, and how to fix it when it breaks.

Contents

1. [What is uv?](#)
2. [Install uv \(and update it later\)](#)
3. [The three files: pyproject.toml, uv.lock, .python-version](#)
4. [The two situations you will be in](#)
5. [Conda → uv cheat sheet](#)
6. [Everyday commands](#)
7. [Updating packages and Python](#)
8. [Managing Python versions](#)
9. [Jupyter, VS Code and Cursor](#)
10. [Ten projects, ten environments?](#)
11. [When something breaks](#)
12. [This course's project](#)
13. [Commands to remember](#)
14. [Simple rules](#)

1. What is uv?

uv is a fast Python package and environment manager. It does the jobs you used **Conda + pip + virtualenv** for, in one tool, and it also downloads Python itself.

The mental model, translated from Conda:

Conda idea	uv equivalent	What it is
A named environment (conda activate myproj)	A .venv/ folder <i>inside the project</i>	The actual installed packages. Lives next to your code, not in a central list.
environment.yml	pyproject.toml	The list of packages you asked for. Short, human-written.
(no standard equivalent)	uv.lock	The exact version of every package that got installed, including the ones you never typed. Machine-written.
conda install python=3.12	.python-version	One line saying which Python this project uses.

Main idea: a uv project is a folder with a pyproject.toml, a uv.lock, and a .venv that uv builds from them. You almost never "activate" anything; you run uv run ... and uv uses the right environment automatically.

2. Install uv (and update it later)

Windows

Open **PowerShell** and run:

```
powershell -ExecutionPolicy Bypass -c  
"irm https://astral.sh/uv/install.ps1  
| iex"
```

Then close and reopen PowerShell. (Alternative:
`winget install astral-sh.uv`)

macOS / Linux

Open **Terminal** and run:

```
curl -LsSf  
https://astral.sh/uv/install.sh | sh
```

Then close and reopen Terminal. (Alternative
on macOS: `brew install uv`)

Check that it worked

```
uv --version
```

Update uv itself

uv releases often. If you installed it with the commands above:

```
uv self update
```

If you installed it with winget or Homebrew instead, update it the same way you installed it: `winget upgrade astral-sh.uv` or `brew upgrade uv`. uv will tell you if `self update` is not the right tool.

Updating uv does **not** change any of your projects. Package versions are decided by `uv.lock`, not by the uv version.

3. The three files, explained

pyproject.toml — "what I want"

A short text file you can read and edit. It names the project, says which Python versions it supports, and lists the packages you asked for. This is the course's real one, trimmed:

```
[project]
name = "df-pycaret-env"
version = "2026.9"
requires-python = ">=3.13,<3.14"      # which Python this project needs
dependencies = [
    "pycaret-core==3.5.0",           # exactly this version
    "statsmodels>=0.14,<0.15",      # any 0.14.x
    "lightgbm>=4.0",                 # 4.0 or newer
    "jupyterlab>=4",
]
```

You do not usually edit this by hand. `uv add pandas` writes the line for you and `uv remove pandas` deletes it. The version rules (`==`, `>=`, `<`) are the same ones `pip` uses.

uv.lock — "what I actually got"

When `uv` installs your dependencies, each one pulls in its own dependencies, and those pull in more. A request for 7 packages can install 165. `uv.lock` records the exact version of *every one of them*, for every operating system, so that a classmate running `uv sync` next month gets byte-for-byte the same environment you have today.

- It is generated by `uv`. **Never edit it by hand.**
- It is long and looks scary. You never need to read it.
- **Commit it to Git.** That is the whole point: the lock file is what makes the project reproducible.
- Conda has nothing like it by default, which is why "it works on my machine" happens more with Conda.

.python-version — "which Python"

One line, for example `3.13`. `uv` reads it and uses (or downloads) that Python for the project. Change it with `uv python pin 3.14`, not with a text editor, so `uv` can check it is valid.

And .venv/ — the built environment

The folder with the installed packages. It is disposable: delete it any time and `uv sync` rebuilds it from the two files above in seconds.

```
my-project/
├── .venv/           ← built by uv, do NOT commit
├── .python-version  ← commit
├── pyproject.toml   ← commit
├── uv.lock          ← commit
└── main.py
```

4. The two situations you will be in

A. Someone gives you a project (this course)

The project already has `pyproject.toml` and `uv.lock`. You do not create anything, you rebuild it:

```
git clone
https://github.com/PJalgotrader/Deep_
forecasting-USU.git
cd Deep_forecasting-
USU/python_migration/uv
uv sync
uv run jupyter lab
```

First `uv sync` also downloads Python 3.13 if you do not have it. Later ones take seconds.

B. You start your own project

You create the files, then add packages one by one:

```
mkdir my-project
cd my-project
uv init --python 3.13
uv add numpy pandas scikit-learn
matplotlib
uv add --dev jupyterlab ipykernel
uv run python
```

`uv init` creates `pyproject.toml`, `.python-version`, a `.gitignore` and a hello-world `main.py`.

Never run `uv init` inside a project that already has a `pyproject.toml`. If the files are there, the only command you need is `uv sync`.

5. Conda → uv cheat sheet

Task	Conda	uv
Create project/environment	<code>conda create -n myproj python=3.13</code>	<code>uv init --python 3.13</code>
Rebuild from a file	<code>conda env create -f environment.yml</code>	<code>uv sync</code>
Activate environment	<code>conda activate myproj</code>	Not needed, use <code>uv run</code>
Add a package	<code>conda install pandas</code>	<code>uv add pandas</code>
Add a dev-only tool	<code>conda install jupyterlab</code>	<code>uv add --dev jupyterlab</code>
Remove a package	<code>conda remove pandas</code>	<code>uv remove pandas</code>
Update one package	<code>conda update pandas</code>	<code>uv lock --upgrade-package pandas</code> then <code>uv sync</code>
Update everything	<code>conda update --all</code>	<code>uv lock --upgrade</code> then <code>uv sync</code>
Run Python	<code>python</code>	<code>uv run python</code>
Run a script	<code>python app.py</code>	<code>uv run python app.py</code>
Install a Python version	Managed by Conda	<code>uv python install 3.13</code>
Change the project's Python	<code>conda install python=3.14</code>	<code>uv python pin 3.14</code> then <code>uv sync</code>
List installed packages	<code>conda list</code>	<code>uv pip list</code> (or <code>uv tree</code> to see who depends on what)
Export the environment	<code>conda env export</code>	Nothing to do: <code>pyproject.toml</code> + <code>uv.lock</code> already are the export

Task	Conda	uv
Delete the environment	<code>conda env remove -n myproj</code>	Delete the <code>.venv</code> folder

6. Everyday commands

```
uv add pandas           # add a package (writes pyproject.toml + uv.lock, installs it)
uv add "pandas>=2.2"    # add with a version rule
uv add --dev pytest ruff # tools you need while developing, not to run the code
uv remove pandas         # the opposite of add
uv sync                  # make .venv match pyproject.toml + uv.lock
uv run python script.py  # run anything inside the project's environment
uv run jupyter lab        # same idea, for Jupyter
uv tree                  # show the dependency tree
```

Prefer `uv add package-name` over `pip install package-name`. `uv add` records the package in the project files so everyone else gets it too. `pip install` changes only your machine and nobody else will know.

If you really want a classic activated shell, it still works: `.venv\Scripts\activate` on Windows, `source .venv/bin/activate` on macOS/Linux. Then plain `python` and `jupyter` point at the project. Most people stop needing this once `uv run` becomes a habit.

7. Updating packages and Python

Because of `uv.lock`, nothing updates by itself. `uv sync` always installs the locked versions. To move forward you change the lock on purpose, then sync.

You want to...	Run
Update one package to its newest allowed version	<code>uv lock --upgrade-package pandas</code> <code>uv sync</code>
Update every package	<code>uv lock --upgrade</code> <code>uv sync</code>
Change the rule for a package (for example allow pandas 3)	<code>uv add "pandas>=3"</code>
Move the project to a newer Python	<code>uv python pin 3.14</code> <code>uv sync</code>
Get a classmate's changes after <code>git pull</code>	<code>uv sync</code>
Update uv itself	<code>uv self update</code>

Careful in a shared project. `uv lock --upgrade` rewrites `uv.lock`. In this course, that file is maintained by the instructor. Run upgrades in your own projects, not in `python_migration/uv`, or you will drift away from what the notebooks were tested on.

8. Managing Python versions

uv downloads Python for you. No python.org installer, no Anaconda needed.

```
uv python list           # what is installed, what is available
uv python install 3.13   # download a version (once, shared by all projects)
uv python pin 3.13       # write .python-version for THIS project
uv run python --version  # confirm which one the project uses
```

Different projects can use different Pythons. The downloaded versions live in one shared place; each project just points at one of them.

9. Jupyter, VS Code and Cursor

Add Jupyter support to a project (already done in the course project):

```
uv add --dev jupyterlab ipykernel
```

Run Jupyter:

```
uv run jupyter lab
```

VS Code / Cursor, notebooks inside the project folder

Open the project folder as your workspace. In a notebook click **Select Kernel** → **Python Environments** and pick the interpreter inside your-project/.venv. VS Code usually finds it on its own.

VS Code / Cursor, notebooks somewhere else (this course)

The course notebooks live in the Module folders, not inside python_migration/uv. Register the environment once as a named kernel, and it shows up in every notebook, wherever it is:

```
cd python_migration/uv
uv run python -m ipykernel install --user --name df-pycaret --display-name "Python (df-pycaret)"
```

Then in any notebook: **Select Kernel** → **Jupyter Kernel** → **Python (df-pycaret)**.

10. What about 10 projects = 10 virtual environments?

Yes. If you have 10 separate projects, you normally have 10 separate `.venv` folders. This is intentional because each project is isolated.

It is not as wasteful as it looks. `uv` keeps a **global package cache** and links files from it instead of downloading and copying everything again for every project. Ten projects that all use `pandas` share one downloaded copy.

```
uv cache dir      # where the cache lives
uv cache prune    # remove entries no project uses any more (safe)
uv cache clean    # delete the whole cache (safe, next sync re-downloads)
```

Why isolation is good: upgrading NumPy, PyTorch, or pandas for Project B will not accidentally break Project A.

11. When something breaks

Symptom	What it means	Fix
ModuleNotFoundError: No module named 'pycaret' but you installed it	You ran a different Python (the system one, or Conda's base), not the project's	Use <code>uv run python ...</code> , or pick the <code>.venv</code> kernel in VS Code
Environment feels broken or confusing	Something got installed by hand, or half-finished	Delete <code>.venv</code> , then <code>uv sync</code> . Nothing is lost: the recipe is in the two files
Still broken after that	Cached package is corrupt	<code>uv sync --reinstall</code>
Windows: Failed to persist temporary file ... os error 3	Folder path too long (Windows 260-character limit)	Move the project to a short path such as <code>C:\dev\my-project</code>
Project lives in OneDrive / Google Drive and everything is slow	The sync client is uploading thousands of <code>.venv</code> files	Keep projects outside synced folders, or exclude <code>.venv</code> from sync
<code>uv add</code> says versions conflict	Two packages cannot agree on a shared dependency	Read the last lines of the message, they name the two packages. Loosen one rule, or leave that package out

Delete the environment

Windows PowerShell:

```
Remove-Item -Recurse -Force .venv
```

macOS / Linux:

```
rm -rf .venv
```

Deleting `.venv` does **not** delete your project or dependency list.

The escape hatch

`uv pip install` something installs into the project's `.venv` the `pip` way, without touching `pyproject.toml` or `uv.lock`. Useful for a quick experiment. Not recorded, so the next `uv sync` may remove it, and classmates will not get it. For anything you want to keep, use `uv add`.

12. This course's project, start to finish

```
# once
cd python_migration/uv
uv sync
uv run python -m ipykernel install --user --name df-pycaret --display-name "Python (df-pycaret)"

# every time
uv run jupyter lab           # or open a notebook in VS Code and pick "Python (df-pycaret)"

# check
uv run python -c "from pycaret.utils import version; print(version())" # 3.5.0
```

Do not run `uv add`, `uv remove`, or `uv lock --upgrade` inside this folder unless the instructor asks. The lock file is what makes your leaderboard match the one in the lecture.

13. The 12 commands to remember

```
uv sync           # build / rebuild the environment from the project files
uv run python app.py      # run inside the environment
uv run jupyter lab
uv init --python 3.13     # start a NEW project
uv add pandas           # add a package
uv add --dev jupyterlab  # add a development tool
uv remove pandas
uv lock --upgrade-package pandas # update one package (then uv sync)
uv python install 3.13
uv python pin 3.13
uv tree              # who depends on what
uv self update       # update uv
```

14. Simple rules

1. One real project → one `.venv`.
2. Got a project with a `uv.lock`? Run `uv sync`, nothing else.
3. Use `uv add` instead of `pip install`.
4. Use `uv run` instead of relying on environment activation.
5. Commit `pyproject.toml`, `uv.lock` and `.python-version`.
6. Do not commit `.venv`. Never edit `uv.lock` by hand.
7. Use `--dev` for Jupyter, testing, and development tools.
8. Updates are deliberate: `uv lock --upgrade...` then `uv sync`.
9. If the environment gets messy, delete `.venv` and run `uv sync`.
10. Keep Conda only when you truly need unusual non-Python/system packages.

Bonus: instruction for AI coding assistants

You can place this in `CLAUDE.md` or `AGENTS.md`:

```
This project uses uv for Python dependency and environment management.  
Use `uv add` / `uv remove` for dependencies and `uv run` for commands.  
Do not use pip or Conda unless explicitly requested. Never edit uv.lock by hand.
```